🛒 Ecommerce Order and Inventory Management

A MySQL project that models an online store end to end: a normalized 5-table schema, a Python-generated dataset of **1,000 customers, 150 products and 3,000 orders**, and **25 SQL queries** for business analysis and data validation.

📌 Highlights

- **5 normalized tables** with primary keys, foreign keys, `UNIQUE` and `NOT NULL` constraints
- **Historical pricing preserved:** `unit_price` is stored on every order line, so past revenue never changes when product prices change
- **Synthetic dataset built with Python:** 1,000 customers across 12 cities, 150 products across 5 categories, 3,000 orders across 4 statuses
- **25 analysis queries** covering joins, aggregations, subqueries, anti-joins and integrity checks
- **Order and inventory insights:** order status mix, monthly trends, top customers and products, low stock alerts, inventory value, payment mode breakdown

---

🗂️ ER Diagram

ER Diagram

Table	Purpose	Key columns
`customers`	Registered buyers	`customer_id`, `name`, `email` (unique), `city`, `signup_date`
`products`	Catalog and stock	`product_id`, `product_name`, `category`, `price`, `stock`
`orders`	One row per order	`order_id`, `customer_id` (FK), `order_date`, `order_status`
`order_items`	Products inside each order	`order_item_id`, `order_id` (FK), `product_id` (FK), `quantity`, `unit_price`
`payments`	Payment records	`payment_id`, `order_id` (FK), `payment_mode`, `amount`, `payment_date`

Relationships: a customer places many orders, an order contains many order items, each order item refers to one product, and an order has payment records.

📊 Dataset

Entity	Records
Customers	1,000
Products	150
Orders	3,000
Order statuses	Delivered, Pending, Shipped, Cancelled
Payment modes	UPI, Card, Net Banking, Cash

Data is generated with a seeded Python script (`random.seed(42)`), so it is reproducible.

🔍 What the Queries Answer

Area	Examples
Data validation	Record counts per table, referential integrity check, orders with no payment
Orders	Orders by status, orders per month, items and units per order
Customers	Customers by city, top customers by order count, customers who never ordered, signups per month
Products	Top 10 products by units sold, demand by category, products never ordered
Inventory	Low stock products, inventory value per product and in total
Payments	Totals by payment mode, payments per month, average payment

All queries are in `ecommerce_cep_analysis_queries.sql`.

🚀 How to Run

1. Create the database:

```sql
CREATE DATABASE ecom_cep;
```

2. Create the tables:

```sql
SOURCE 01_create_tables.sql;
```

3. Generate the CSV files (saved in `data/`):

```bash
python generate_data.py
```

4. Import the CSVs into the matching tables (MySQL Workbench: *Table Data Import Wizard*).
5. Verify the structure and run the analysis:

```sql
SOURCE 07_check_table_structure.sql;
SOURCE ecommerce_cep_analysis_queries.sql;
```

📁 Project Structure

```
├── README.md
├── er_diagram.jpg                      # ER diagram image
├── ER_diagram.mwb                      # MySQL Workbench model
├── 01_create_tables.sql                # Schema (DDL)
├── 07_check_table_structure.sql        # DESCRIBE all tables
├── generate_data.py                    # Synthetic data generator
└── ecommerce_cep_analysis_queries.sql  # 25 analysis and validation queries
```

⚠️ Limitations

- The data is synthetic, so the results illustrate the analysis and do not reflect a real business.
- The `payments` table has no `payment_status` column, so payment totals mean amounts recorded, not confirmed revenue.

- Order dates and customer signup dates are generated independently, so an order can predate its customer's signup.
- Product prices are random within a wide range and are not tuned per category.

🔮 Future Improvements

- Revenue analysis by month, category and city
- Window functions (e.g. `RANK()` for top customers per city), CTEs and views
- `CHECK` constraints and a `payment_status` column
- Automated data loading with `LOAD DATA INFILE`

🛠️ Tech Stack

MySQL 8 · MySQL Workbench · Python (`csv`, `random`, `datetime`)